

TECHNICAL TRAINING COURSE

Web Services for Java EE

- JAX-WS · JAXB · SAAJ · JAXP
- JAX-RS for RESTful Services
- Web Services Metadata (JSR 181)
- Provider & Dispatch APIs · JAXR

2-Day Course | Intermediate – Java EE Developer / Architect



COURSE AGENDA

9 Sections — 2 Days — Code Labs

01 Scenarios for Hosting Web Services

02 Web Services in Java EE

03 JAX-WS and JAXB

04 Web Services Metadata

05 WSDL to Java and Java to WSDL

06 Provider and Dispatch APIs

07 SAAJ and JAXP

08 JAX-RS for RESTful Services

09 JAXR

Scenarios for Hosting Web Services

1

B2B Integration

Partners exchange orders/invoices over HTTPS using SOAP+WS-Security

2

Internal SOA

Monolith decomposed — EJB endpoints consumed by portal + batch

3

Legacy Wrapping

COBOL/C++ systems exposed via JAX-WS facade + JCA adapter

4

RESTful Exposure

Mobile/browser clients access product data via JAX-RS JSON API

Deployment Topologies

Standalone

`SE + Endpoint.publish()`

Dev / Testing only

Servlet Container

`Tomcat / Jetty + CXF/Metro`

Lightweight SOAP/REST

EJB Container

`GlassFish / WildFly`

Full Java EE — transactions, pooling, security

Managed Cluster

`WebSphere / WebLogic`

Enterprise HA & load balancing

EJB endpoints gain JTA transactions, JNDI, DI, lifecycle management, and pooling automatically — choose EJB for production services.

Web Services in Java EE

The complete API stack

JAX-WS 2.x

SOAP endpoints & clients

JAXB 2.x

Java ↔ XML binding

SAAJ 1.3

Low-level SOAP messages

JAXP

SAX / DOM / StAX / XSLT

JAX-RS 2.x

REST resource classes

JSR 181

@WebService annotations

JSR 109

Deployment & Port Components

JAXR 1.0

Registry client (legacy)

JSR 109 — The Port Component Model

`@WebService`
`@Stateless`
`@HandlerChain`

Port Component

Service Endpoint Interface (SEI)

Java contract — method signatures

Service Implementation Bean (SIB)

Implementing class (@Stateless etc.)

WSDL Port

Concrete binding + network address

Handler Chain

Optional interceptor pipeline

**WSDL
Contract**

JAX-WS — Core Annotations

@WebService

Marks class as SOAP endpoint; sets service/port name & namespace

@WebMethod

Marks method as WSDL operation; operationName or exclude=true

@WebParam

Controls XML element name & mode (IN/OUT/INOUT) of a parameter

@WebResult

Controls XML element name of the return value

@SOAPBinding

Sets style (DOCUMENT/RPC) & use (LITERAL/ENCODED)

@HandlerChain

Associates a handler chain XML file with endpoint or client

@Oneway

One-way operation — must return void; no response message

JAXB — Java ↔ XML Binding

```
@XmlElement(name="customer")
@XmlType(propOrder={"id","name","tier"})
public class Customer {

    @XmlElement(required=true)
    private String id;

    @XmlElement(required=true)
    private String name;

    @XmlElement(defaultValue="STANDARD")
    private String tier;

    @XmlAttribute(name="currency")
    private String currency="GBP";

    // getters/setters...
}
```

@XmlElement

Top-level XML element

@XmlElement

Maps field to XML element

@XmlAttribute

Maps field to XML attribute

@XmlType

Sets element order (propOrder)

@XmlTransient

Excludes field from XML

@XmlEnum

Maps Java enum to XSD enum

Web Services Metadata

JSR 181 — Turn any POJO into a web service endpoint with annotations

@SOAPBinding — Binding Styles

Combination	WS-I Compliant	Recommended
DOCUMENT / LITERAL / WRAPPED	✓ Yes	✓ Best choice
DOCUMENT / LITERAL / BARE	✓ Yes	Usable
RPC / LITERAL	✓ Yes	Avoid
RPC / ENCODED	✗ No	✗ Never use

@HandlerChain: SOAPHandler (full SOAP envelope) | LogicalHandler (payload only — protocol-independent)

WSDL-First vs Java-First Development Paths

WSDL-First (Contract-First)

- Start: WSDL document
- Tool: wsimport
- Generates: SEI, stubs, value types
- Contract stable before implementation
- Best for: External partners / multiple impls

```
wsimport -keep -p com.example.client  
CustomerService.wsdl
```

Java-First (Code-First)

- Start: @WebService annotated class
- Tool: wsgen
- Generates: WSDL + XSD
- Contract derived from code
- Best for: Internal / rapid prototyping

```
wsgen -cp target/classes -wsdl  
com.example.CustomerServiceImpl
```

Using wsimport-Generated Client Code

```
// 1. Service object - reads WSDL annotation
CustomerService service = new CustomerService();

// 2. Get port proxy (implements SEI)
CustomerPortType port = service.getCustomerPort();

// 3. Invoke operation - JAXB handles marshalling
GetCustomerRequest req = new GetCustomerRequest();
req.setCustomerId("CUST-001");
GetCustomerResponse resp = port.getCustomer(req);
System.out.println(resp.getCustomer().getName());

// 4. Override endpoint address at runtime
BindingProvider bp = (BindingProvider) port;
bp.getRequestContext().put(
    BindingProvider.ENDPOINT_ADDRESS_PROPERTY,
    "https://api.example.com/CustomerService");
```

Provider & Dispatch APIs

Dynamic message-level access — no generated stubs required

Provider<T> — Service Side

Provider<SOAPMessage>

Full SOAP envelope | Mode.MESSAGE

Provider<Source>

Body payload only | Mode.PAYLOAD

Provider<DataSource>

Binary MIME | MTOM/SwA scenarios

Dispatch<T> — Client Side

invoke (T)

Synchronous — blocks until response

invokeAsync (T)

Async polling — returns Response<T>

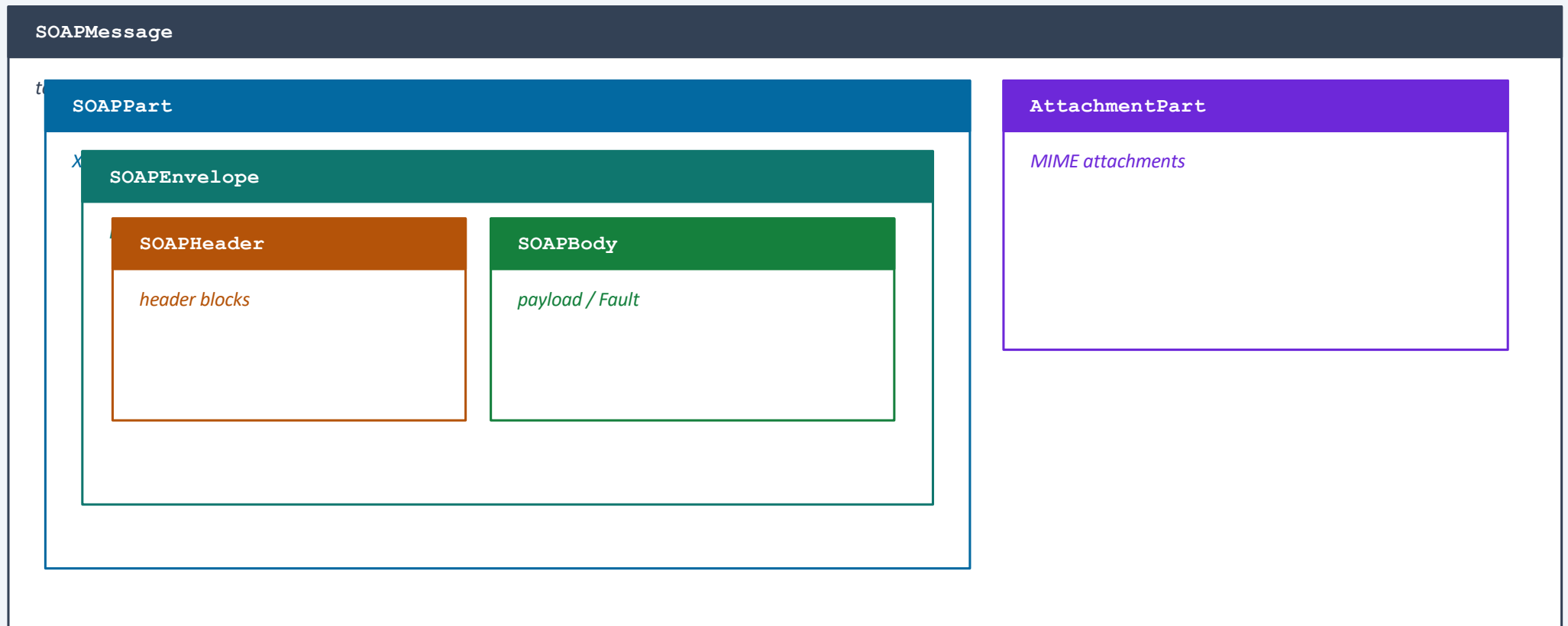
invokeAsync (T, Handler)

Async callback — fires when done

invokeOneWay (T)

Fire-and-forget — no response

SAAJ — SOAP Message Object Model



JAXP — XML Processing APIs

SAX	Event-driven <i>Large docs; sequential read</i>	Memory: Streaming	<code>SAXParserFactory</code>
DOM	Tree-based <i>Random access; edit & write</i>	Memory: In-memory	<code>DocumentBuilderFactory</code>
StAX	Pull-based <i>High perf; better than SAX</i>	Memory: Streaming	<code>XMLInputFactory</code>
XSLT	Transform <i>XML→XML/HTML/text transform</i>	Memory: Varies	<code>TransformerFactory</code>

JAX-RS for RESTful Services

JSR 370 — Annotation-driven REST resources in Java EE

@Path

Binds URI path template to class/method — supports {param}

@PathParam

Injects URI template variable into parameter

@GET/@POST/etc.

HTTP method binding annotations

@QueryParam

Injects query string value (?key=val) into parameter

@Produces

Declares media types the method returns (response Content-Type)

@Context

Injects UriInfo, HttpHeaders, SecurityContext, Request

@Consumes

Declares media types the method accepts (request Content-Type)

@ApplicationPath

Sets base URI for all resources in the Application class

JAX-RS Resource Class — Full Example

```
@Path("/customers")
@Produces(APPLICATION_XML)
@Consumes(APPLICATION_XML)
public class CustomerResource {

    @Inject CustomerService svc;

    @GET // GET /customers
    public List<Customer> list(
        @QueryParam("tier") String tier) {
        return svc.find(tier);
    }

    @GET @Path("{id}") // GET /customers/{id}
    public Response get(@PathParam("id") String id){
        Customer c = svc.findById(id);
        if(c==null) throw new NotFoundException();
        return Response.ok(c).build();
    }

    @POST // POST /customers -> 201
    public Response create(Customer c,
        @Context UriInfo ui) {
        Customer saved = svc.create(c);
        URI loc = ui.getAbsolutePathBuilder()
            .path(saved.getId()).build();
        return Response.created(loc).entity(saved).build();
    }
}
```

HTTP Verbs & Status Codes

GET

200 OK / 404

POST

201 Created

PUT

200 OK / 404

DELETE

204 No Content

```
@ApplicationPath("/api")
class App extends Application {}
```


JAXR — Java API for XML Registries

JAXR Object	Description
Organization	Business entity (≈ UDDI businessEntity)
Service	Web service offered by an Organization
ServiceBinding	Technical endpoint (≈ UDDI bindingTemplate)
SpecificationLink	Link to WSDL tModel / technical spec
Classification	Associates an object with a taxonomy value
tModel / ClassificationScheme	Technical model / taxonomy definition

Deprecated in Java
EE 7

Removed from
Jakarta EE

Maintain legacy
apps only

Modern alternatives: Consul · Eureka · AWS API Gateway · Backstage · Apache jUDDI

API Quick Reference — Web Services for Java EE

JAX-WS

JSR 224

SOAP endpoints & clients**JAXP**

JAXP 1.4

SAX/DOM/StAX/XSLT**JAXB**

JSR 222

Java ↔ XML marshalling**JSR 181**

JSR 181

@WebService annotations**JAX-RS**

JSR 370

RESTful resource classes**JSR 109**

JSR 109

Deployment & Port Components**SAAJ**

JSR 67

Low-level SOAP construction**JAXR**

JSR 93

Registry client (legacy)

Key Takeaways

01

Java EE containers provide transactions, pooling, DI and security to any hosted web service.

02

JAX-WS + JAXB covers SOAP endpoints. Use document/literal/wrapped for WS-I compliance.

03

JSR 181 annotations (`@WebService`, `@SOAPBinding`) drive WSDL generation with no XML config.

04

Choose WSDL-first for partner integrations; Java-first for internal or rapid-prototyping.

05

`Provider<T>` / `Dispatch<T>` give raw SOAP access for routing, testing, and protocol work.

06

JAX-RS delivers REST APIs with `@Path`, `@GET`/`@POST`, `@Produces`, `@Consumes` and CDI injection.

Thank You

Questions & Lab Time

Next steps:

- Lab exercises for each section
- End-of-course quiz (30 questions)
- Design exercise: build a JAX-WS + JAX-RS API
- Pass mark: 70% for certification

